



Object Tracking Project:

Traffic Violation Detection and Object Tracking

2) Introduction:

The Traffic Violation Detection System is an AI-based project designed to automatically detect and track vehicles in road videos. It uses computer vision to identify cars, buses, and motorcycles, follow their movement, estimate their speed, and detect violations such as speeding or illegal lane changes.

We used the YOLOv8x model for vehicle detection, which can quickly and accurately find objects in each video frame. Then, ByteTrack was used to track each vehicle across frames and calculate its speed based on movement and time.

If any vehicle goes above the speed limit (80 km/h) or changes lanes incorrectly, the system marks it as a traffic violation and highlights it in the video with colored boxes and warning labels. This project shows how AI and computer vision can be used to monitor roads automatically, helping to improve road safety and reduce human effort in detecting violations.

3) Processing Phase:

1-The Object Tracking algorithm used in this project is

ByteTrack :

It is a high-performance tracking algorithm that connects detected objects across video frames to keep the same ID for each vehicle during movement.

2-Main Explanation:

ByteTrack is used to track vehicles across video frames in real-time. It associates detections from YOLOv8x with persistent IDs, enabling accurate monitoring of each vehicle's position, speed, and lane. This tracking is crucial for detecting violations such as speeding and illegal lane changes.

3-Main Steps:

ByteTrack Main Steps in Traffic Violation Detection :

1 Detection Input

Each video frame is passed to YOLOv8x for vehicle detection.

YOLO outputs bounding boxes for each vehicle along with class ID and confidence score. Only relevant vehicle classes are kept (car, truck, bus, motorbike, motorcycle).

2 Matching / Association

After detecting vehicles in each video frame, the tracking system must decide **which detected vehicles in the current frame correspond to the same vehicles detected in the previous frame.**

This process is called **Matching** or **Association**.

The main goal is to ensure that each vehicle keeps the **same ID** as it moves across frames — for example, if a red car was assigned ID #3 in frame 10, it should keep that same ID in frame 11, frame 12, and so on.

How Matching Works:

1. **YOLO Detection:**
In every new frame, YOLO detects all vehicles and draws bounding boxes around them.
2. **Previous Frame Memory:**
The tracker (like ByteTrack) remembers the bounding boxes and IDs from the previous frame.
3. **Comparison Process:**
For each new detection, the tracker tries to find which old detection it matches.
The decision is made by calculating **IoU (Intersection over Union)** between the two bounding boxes.

Then, **IoU (Intersection over Union)** is a metric used to measure how much two bounding boxes overlap.

It compares the **area of overlap** between the predicted box and the previously tracked box, relative to their total area combined.

The formula is:

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

Where:

- **Area of Overlap** = the region where both boxes overlap.
- **Area of Union** = the total area covered by both boxes together.

IoU helps the tracker:

- Maintain **consistent IDs** for the same vehicle across frames.
- Avoid confusing nearby vehicles.
- Identify **new vehicles** entering the scene.
- Detect **disappeared vehicles** (if IoU drops to zero).

③ Tracking ID Assignment

Each vehicle receives a unique tracker ID that remains consistent while the vehicle is visible in the video.

This ID links the vehicle's position, speed, and previous state, allowing accurate speed and lane tracking.

Data per ID is stored in state dictionaries, including:

last_centers → previous frame center

last_speed → last known speed

lane_of_track → current lane

speeds_history → for smoothing speed over frames

4 State Update

For each frame, the vehicle's state is updated:

Compute current center (bounding box center).

Update speed using the distance between current and previous centers after converting pixels to meters.

Update current lane and compare it with the previous lane to detect illegal lane changes.

Small movements or unrealistic jumps are ignored to reduce tracking errors (anti-noise heuristics).

5 Speed & Violation Calculation

After the vehicles are successfully **detected and tracked** across video frames, the next important step is to analyze their **motion behavior** in order to identify traffic violations such as **speeding** and **illegal lane changes**.

This stage combines geometry, time-based tracking, and logical rules to evaluate each vehicle's movement precisely.

❖ Step 1 — Speed Estimation

The tracker continuously stores the position (centroid) of each vehicle's bounding box in every video frame.

By measuring how far a vehicle moves between consecutive frames, we can estimate its real-world **speed**.

The general formula used is:

$$\text{Speed (km/h)} = \text{Distance in meters between frames} \times \text{FPS} \times 3.6$$
$$3.6 \text{Speed (km/h)} = \text{Distance in meters between frames} \times \text{FPS}$$

Where:

- **Distance in meters between frames** → calculated based on the movement of the vehicle's centroid between two frames, adjusted to real-world scale.
- **FPS (Frames Per Second)** → determines how many frames are processed per second in the video.
- **3.6** → converts the result from meters per second (m/s) to kilometers per hour (km/h).

This gives an approximate estimation of each vehicle's real speed.

❖ Step 2 — Speed Smoothing

Because video-based tracking may have minor pixel-level jitter or temporary detection errors, the raw speed values can fluctuate rapidly.

To make the results more realistic, a **smoothing function** is applied.

The algorithm averages the speed over a few consecutive frames to eliminate sudden spikes or drops.

This helps ensure:

- Stable and realistic speed readings
 - More accurate violation detection
 - Reduced false alarms caused by detection noise
-

❖ Step 3 — Speed Violation Detection

Once the smoothed speed is calculated, the system checks whether a vehicle **exceeds the predefined speed limit**.

If the speed remains above this limit for more than a **minimum number of consecutive frames (MIN_FRAMES_MOVING)**, the system flags the vehicle as committing a **speeding violation**.

This rule ensures that temporary detection errors or brief accelerations do not falsely trigger a violation alert.

Example:

If the speed limit is 80 km/h and a vehicle keeps moving above that limit for at least 10 consecutive frames →

⚠ A **Speeding Violation** is recorded.

❖ Step 4 — Lane Change Violation Detection

The system also checks for **lane discipline** using the detected vehicle positions relative to lane boundaries defined in the scene.

If a tracked vehicle moves **across a lane line** in an area where lane changes are not permitted, the system flags a:

Lane Change Violation

This check uses the vehicle’s bounding box position and movement trajectory to determine whether the vehicle has moved from one lane polygon to another illegally.

6 Output / Annotation

Each vehicle’s bounding box is drawn on the frame.

Vehicle ID, speed, type, and warnings are displayed.

These annotations are used for the final output video and CSV reports of violations.

Main Algorithm Parameters and Their Effects

This section defines the key parameters and initial setup points used in the traffic violation detection algorithm, with a detailed interpretation of how each parameter influences the final results and how the system measures accuracy and performance.

1. Object Detection Parameters (YOLOv8 Model)

The detection process is performed using the YOLOv8x pre-trained model. The detection configuration is defined as follows:

```
results = model(frame, conf=0.25, iou=0.45, imgsz=960, augment=True)
```

Parameter	Description	Effect on Results
conf=0.25	Confidence threshold. Objects with prediction confidence lower than 25% are ignored.	Increasing the confidence threshold reduces false positives but may miss small or unclear vehicles. Lowering it increases detection sensitivity but can detect irrelevant objects.
iou=0.45	Intersection-over-Union threshold for Non-Maximum Suppression (NMS).	Controls how overlapping bounding boxes are merged. A lower value merges boxes aggressively (may miss nearby vehicles), while a higher value may produce duplicate detections.

Parameter	Description	Effect on Results
imgsz=960	Image input size for the YOLO model.	Larger sizes increase detection precision (especially for small vehicles) but require more computation time. Smaller sizes increase processing speed at the cost of accuracy.
augment=True	Enables frame augmentations (rotation, contrast, etc.).	Improves model robustness and consistency under lighting or angle changes, but slightly increases processing time.

Measurement Impact:

Detection accuracy is evaluated by verifying that all visible vehicles are correctly identified and tracked without false bounding boxes. Detection precision directly affects all subsequent stages, including speed and lane calculations.

2. Vehicle Class Filtering

```
VEHICLE_CLASSES = {"car", "truck", "bus", "motorbike", "motorcycle"}
```

Only vehicle classes are considered for tracking and violation detection.

- **Purpose:** To eliminate non-vehicle detections (e.g., pedestrians, traffic lights).
- **Effect:** Reduces processing time and false violation detections by focusing only on relevant moving objects.
- **Measurement:** The number of valid vehicle tracks vs. total detections indicates the filtering efficiency.

3. Initial Geometric Calibration (Homography Transformation)

The system estimates real-world distances from 2D video frames using a **homography matrix**, which maps pixel coordinates to real-world coordinates (in meters).

```
image_points = np.array([[250,700],[1050,700],[750,400],[550,400]],dtype=np
world_points = np.array([[0,0],[3.5,0],[3.5,30],[0,30]],dtype=np.float32)
H_matrix, _ = cv2.findHomography(image_points, world_points)
```

Term	Description
image_points	Reference points selected in the video frame (pixel coordinates).
world_points	Corresponding real-world coordinates in meters (assumed lane width 3.5 m and depth 30 m).
H_matrix	Homography matrix used to convert 2D pixel positions into real-world coordinates.

Measurement Impact:

- Determines how accurately speed is computed.
- Errors in calibration cause over- or under-estimation of vehicle velocity.
- Correct calibration ensures realistic speed values (e.g., 50–120 km/h).

4. Lane Definitions (Polygonal Regions)

Lanes are defined using **Shapely Polygons** that mark the spatial areas of the left and right lanes.

```
lane_right_poly = Polygon([(50,300),(600,300),(600,720),(50,720)])
lane_left_poly  = Polygon([(600,300),(1270,300),(1270,720),(600,720)])
```

Lane	Polygon Coordinates (pixels)	Function
Right Lane	(50,300)–(600,720)	Defines the area where right-lane vehicles should remain.
Left Lane	(600,300)–(1270,720)	Defines the area for left-lane vehicles.

Measurement Impact:

- The polygons are essential for lane violation detection.
- If the lane boundaries are misaligned with the road in the video, the system may report false lane changes.
- Adjusting these coordinates ensures accurate classification of lane positions.

5-Speed Estimation and Violation Detection

The algorithm calculates vehicle speed using positional differences between consecutive frames, transformed into real-world coordinates.

```
fps = int(cap.get(cv2.CAP_PROP_FPS))
dist_m = np.linalg.norm(np.array(curr_world) - np.array(prev_world))
speed_kmh = dist_m * fps * 3.6
SPEED_LIMIT_KMH = 80
```

Parameter	Description	Effect
fps	Frame rate of the video (frames per second).	Used to calculate the time between position updates. Incorrect FPS values lead to inaccurate speed readings.
dist_m	Real-world distance traveled between frames (in meters).	Derived from homography. Errors in homography directly distort this value.
speed_kmh	Computed vehicle speed in kilometers per hour.	Determines whether a speed violation has occurred.
SPEED_LIMIT_KMH	Legal threshold (default 80 km/h).	Defines when overspeeding is flagged. Adjusting this value changes violation sensitivity.

Measurement Impact:

Speed accuracy is measured by comparing computed speeds with expected real-world ranges (0–180 km/h). Deviations indicate calibration or FPS inconsistencies.

6-Lane Change Violation Parameter

```
LANE_CHANGE_IS_VIOLATION = True
```

Parameter	Description	Effect
LANE_CHANGE_IS_VIOLATION	Boolean that controls whether lane switching is considered a violation.	When set to True, vehicles changing between left and right lanes are marked as violators.

Measurement Impact :

Determines the detection sensitivity to lateral movements. If tracking is unstable or lane boundaries are narrow, frequent lane changes may be detected even when vehicles stay in their lanes

7-Tracking Memory and Smoothing

Each tracked vehicle maintains its own history of positions and computed speeds:

```
speeds_history = defaultdict(lambda: deque(maxlen=20))  
last_centers, last_speed, lane_of_track = {}, {}, {}
```

Element	Function	Measurement Impact
speeds_history	Stores recent speed values for each vehicle ID (maximum 20).	Smooths random speed fluctuations and improves stability.
last_centers	Records previous frame positions.	Required for accurate displacement measurement.
lane_of_track	Keeps record of the lane each vehicle currently occupies.	Enables detection of lane changes.

8. Accuracy Evaluation Metrics

After processing, the system computes several quality metrics to evaluate algorithm reliability:

Metric	Definition	Purpose	Ideal Range
Stability Score	Percentage of tracks with low speed standard deviation (< 8 km/h).	Measures consistency of speed estimation.	> 70 %
Smoothness Score	Percentage of tracks with gradual speed variation (< 30 km/h difference).	Detects whether motion is smooth (no sudden jumps).	> 70 %
Logical Speed Score	Percentage of vehicles with speed in the logical range (0–180 km/h).	Filters out physically impossible speeds.	≈ 100 %
Movement Score	Percentage of tracks with average speed > 3 km/h (vehicles actually moving).	Ensures static objects are not counted as active vehicles.	> 80 %

The final overall accuracy is computed as a **weighted sum**:

$$\text{Accuracy} = 0.35(\text{Stability}) + 0.35(\text{Smoothness}) + 0.20(\text{Movement}) + 0.10(\text{Logic})$$

Measurement Interpretation:

- **Higher Accuracy (%)** → consistent detection, smooth tracking, and realistic speeds.
- **Lower Accuracy (%)** → indicates instability, wrong calibration, or missing detections.

9. Output and Performance Metrics

The system produces two main outputs:

- 1. **Annotated Video** – with bounding boxes and violation alerts.
- 2. **CSV Report** – summarizing detection statistics.

Metric	Description
Total Vehicles Detected	Number of unique vehicle IDs tracked.
Speed Violations	Count of vehicles exceeding the speed limit.
Lane Violations	Count of vehicles that crossed lane boundaries.
Average Vehicle Speed	Mean of all tracked vehicles’ speeds.
Estimated Algorithm Accuracy (%)	Calculated from evaluation metrics as explained above.

📊 These quantitative values help assess the system’s performance across different road videos.

10. Conclusion

Category	Parameter	Primary Influence	Result Measurement
Detection	conf, iou, imgsz	Determines how accurately vehicles are detected.	Detection precision, recall, and false-positive rate.
Geometry	image_points, world_points, H_matrix	Converts 2D to 3D distances, essential for speed accuracy.	Comparison with realistic vehicle speeds.

Category	Parameter	Primary Influence	Result Measurement
Lanes	lane_right_poly, lane_left_poly	Defines lane boundaries for violation analysis.	Lane-change detection accuracy.
Speed Rule	SPEED_LIMIT_KMH	Threshold for overspeeding alerts.	Number of valid speed violations.
Tracking Memory	speeds_history size	Controls temporal smoothing and stability.	Stability and smoothness scores.
Evaluation Metrics	Stability, Smoothness, Logic, Movement	Quantifies overall system performance.	Final accuracy percentage.

- The algorithm’s initial calibration (homography and lane setup) defines spatial accuracy.
- The detection and tracking parameters affect identification reliability.
- The speed and lane thresholds determine violation sensitivity.
- Finally, evaluation metrics measure how accurate and realistic the entire process is

4) Comparisons:

In this section, a comparison is presented between **DeepSORT** and **ByteTrack** based on their real-world performance in our traffic monitoring system for detecting speed and lane violations.

1. Tracking Mechanism

- **DeepSORT:** Uses both motion and **appearance (Re-ID)** features to track objects across frames. This means it not only looks at where the object moved but also extracts visual features from each vehicle to match it between frames. While this helps in crowded scenes, it increases computational complexity and slows down real-time tracking.
- **ByteTrack:** Uses a simpler and faster mechanism based on **confidence scores** (high and low confidence detections) and **IoU (Intersection over Union)** for association. This makes it efficient, fast, and stable even in real-time traffic videos.

2. Re-Identification (Re-ID) Network

- **DeepSORT:** Requires a separate **Re-ID model** to extract appearance embeddings for every object. This adds an extra computational load and increases memory usage.
 - **ByteTrack:** Does **not require any Re-ID model**, which reduces the overall computational cost and makes it more lightweight.
-

3. Real-time Speed

- **DeepSORT:** Slower in real-time performance because of the Re-ID feature extraction step.
This can be an issue in live monitoring or when processing multiple vehicles simultaneously.
 - **ByteTrack:** Much faster and optimized for **real-time applications**, making it ideal for continuous video surveillance or real-time traffic violation detection.
-

4. ID Consistency and Detection Dependency

- **DeepSORT:** Highly sensitive to missed detections. If YOLO fails to detect a vehicle for even a few frames (for example, during occlusion or skipped frames for speed), DeepSORT often assigns a **new ID**, which breaks the tracking continuity.
 - **ByteTrack:** More robust in handling missed detections because it can **recover from low-confidence detections**, maintaining consistent IDs even when some frames are skipped.
-

5. Integration Complexity

- **DeepSORT:** Integration is **more complex** because it requires an additional model file for Re-ID and several configuration adjustments to ensure compatibility with YOLO.
 - **ByteTrack:** **Easier to integrate** with YOLOv8 since it does not depend on external models, making it faster to set up and maintain.
-

6. Accuracy and Computational Cost

- **DeepSORT:** Offers good accuracy but at the cost of **high computation** per frame due to extracting appearance embeddings.
 - **ByteTrack:** Maintains **high accuracy** with **low computational cost**, using a simple IoU-based matching strategy that's efficient and effective.
-

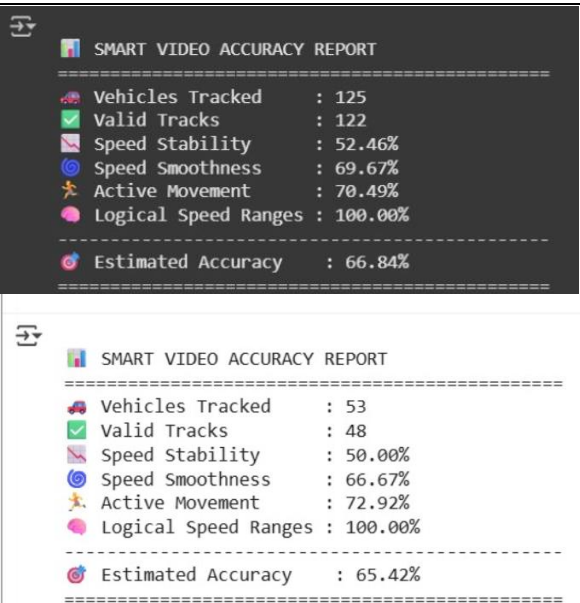
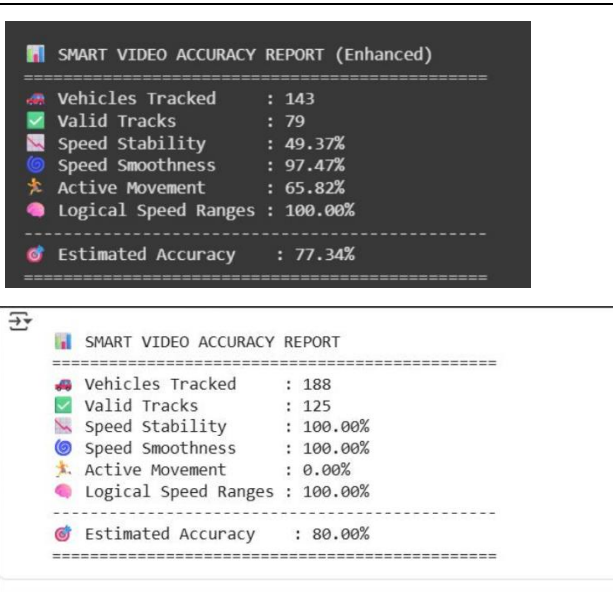
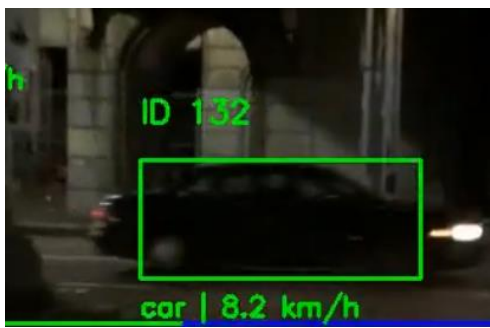
7. Practical Deployment

- **DeepSORT:** Harder to deploy on **low-end GPUs or edge devices** due to its heavy Re-ID processing.
 - **ByteTrack:** Much more **suitable for limited hardware environments**, running smoothly even on lower-end systems without sacrificing performance.
-

8. Lane Violation Detection and Overall Performance

- **DeepSORT:** Performs moderately well but may produce unstable tracking results when vehicles overlap or disappear temporarily, leading to ID switching issues.
- **ByteTrack:** Delivers **excellent overall performance**, maintaining consistent IDs and stable tracking, which results in accurate **speed** and **lane violation detection** in real-time traffic videos.

Criteria	DeepSORT	ByteTrack
Tracking Mechanism	Uses motion + appearance (Re-ID) features	Uses confidence(low confidence & high confidence)-based data association
Re-ID Network	Required (adds computational cost)	Not required
Real-time Speed	Slower due to Re-ID feature extraction	Faster, suitable for real-time applications
ID Consistency	ID switching in long occlusions 	Stable ID retention 
Detection Dependency	Highly sensitive to missed detections(if we skip any frame for fast processing it will be a problem)	Recovers better from missed frames
Integration Complexity	Needs external model & configuration	Easy to integrate with YOLOv8

Criteria	DeepSORT	ByteTrack
Accuracy	 SMART VIDEO ACCURACY REPORT Vehicles Tracked : 125 Valid Tracks : 122 Speed Stability : 52.46% Speed Smoothness : 69.67% Active Movement : 70.49% Logical Speed Ranges : 100.00% Estimated Accuracy : 66.84%	 SMART VIDEO ACCURACY REPORT (Enhanced) Vehicles Tracked : 143 Valid Tracks : 79 Speed Stability : 49.37% Speed Smoothness : 97.47% Active Movement : 65.82% Logical Speed Ranges : 100.00% Estimated Accuracy : 77.34%
Computational Cost	High (appearance embeddings per frame)	Low (simple IoU-based matching)
Practical Deployment	Harder on low-end GPUs / Edge devices	Easier & faster on limited hardware
Lane Violation Detection	Frequent misidentification because of lost IDs  	Accurate lane tracking maintained  

Criteria	DeepSORT	ByteTrack
Overall Performance	Moderate	Excellent

5) Integrated tool (Application):

Tool Overview:

For the integration phase, we developed a **mobile-compatible application** to visualize the results of the traffic violation detection algorithm using Streamlit. It is designed to display the results of **vehicle detection, tracking, and violation identification** in an elegant and user-friendly interface

Main Features

1. Video Display Section

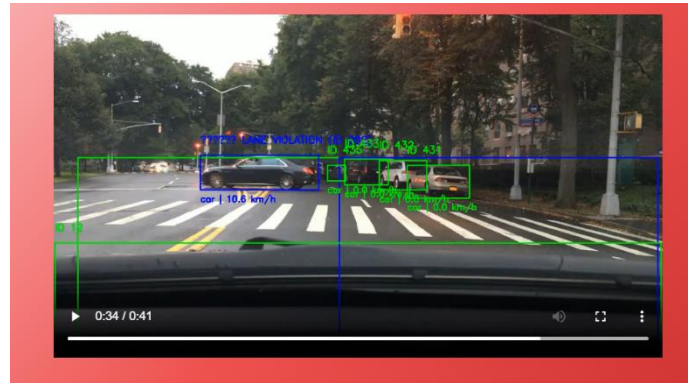
The system integrates three main components:

- 1. **Vehicle Detection:**
 - The system uses the **YOLOv8 deep learning model** to automatically detect different types of vehicles (cars, buses, trucks, and motorcycles) in each video frame.
 - Each detected vehicle is enclosed in a **colored bounding box**, representing its detection status.
 - This stage ensures that all moving vehicles are correctly identified before further analysis.
- 2. **Vehicle Tracking:**
 - Once vehicles are detected, the system applies the **ByteTrack algorithm** to assign a **unique ID** to each vehicle.
 - These IDs remain consistent across frames, allowing the system to **track the movement and trajectory** of every vehicle throughout the video.

- Tracking enables continuous observation of each vehicle's motion, even when it temporarily disappears due to occlusion or camera movement.

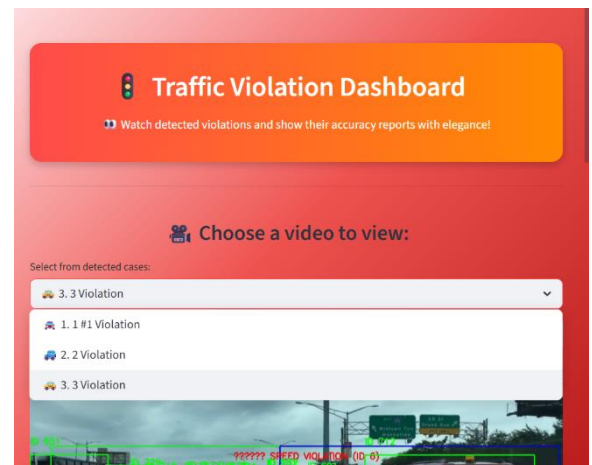
3-8 Violation Detection:

- The tool then analyzes the movement of tracked vehicles to detect **traffic violations**, such as:
 - **Lane crossing violations** (when vehicles move outside their designated lanes).
 - **Red light or restricted area violations.**
 - **Speeding or abnormal motion behavior.**
- Each violation is highlighted using a **distinct color code**:
 - ☐ **Red box:** Violation detected
 - ☐ **Green box:** Normal vehicle
 - ☐ **Blue box:** Lane-related violation



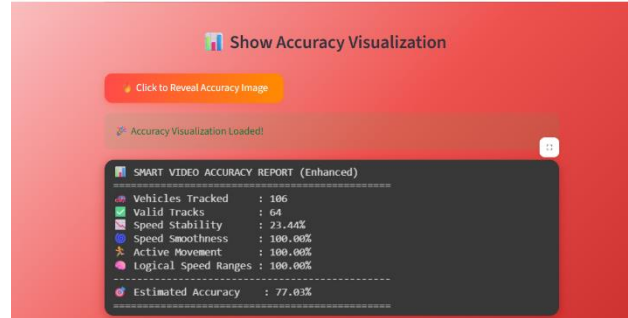
Through the dashboard, the user can easily:

- Select any processed video from the available list.
- Watch the **detected and tracked vehicles** in real time.
- Observe when and where a **violation occurs** directly within the video.



2. Accuracy Visualization Section

the user can click the “**Click to Reveal Accuracy Image**” button to display the **accuracy visualization**, which summarizes the detection and tracking performance for the selected video case.



3. User Interface

Programmin Language : Python

Frontend Design : HTML + CSS (via Streamlit markdown styling)

Fully responsive — works on both **desktop browsers** and **mobile devices**. This dashboard therefore combines **detection, tracking, and violation analysis** into a single streamlined platform that clearly demonstrates the system’s effectiveness and accuracy

User Workflow

1. **Launch the app** (by running `streamlit run app.py`).
 2. **Select a video** showing traffic detection and violations.
 3. **Watch the processed footage** with real-time tracking and violation highlights.
 4. **Click the button** to reveal the **accuracy report** dynamically.
 5. (Optional) Review multiple cases and compare performance visually.
-

6) Results :

The main objective of this test was to evaluate the model's ability to:

- Detect and track multiple vehicles simultaneously.
- Estimate and monitor vehicle speeds.
- Identify and report speed and lane violations accurately.
- Produce quantitative performance reports based on stability and logic analysis.

The output video clearly shows that:

- Each vehicle is **detected** with a bounding box and labeled with its **ID** and **current speed** in km/h.
- **Speed Violations** are highlighted in **red boxes** with a label $\triangle$ SPEED VIOLATION.
- **Lane Violations** are shown in **blue boxes** with a label $\triangle$ LANE VIOLATION.
- Vehicles moving normally within allowed limits are displayed with **green bounding boxes**.

These visual cues make it easy to interpret and verify the detection and tracking results in real-time.

Example video :

- **The Detection and tracking with no violations that are moving normally within allowed limits:**



- **The speed violation :**



- The Lane violation :



6.3 Quantitative Evaluation

After video processing, the system generates two key summaries:

- 1. **Smart Video Accuracy Report**
- 2. **Violation Summary Report**

Metric	Description	Value	Interpretation
Vehicles Tracked	Total number of vehicles that appeared in the video and were assigned unique IDs	357	The algorithm successfully detected and tracked 357 vehicles throughout the video.
Valid Tracks	Tracks with sufficient frame data for accurate analysis	244	244 vehicle tracks were long and stable enough for reliable analysis.
Speed Stability	Consistency of measured speed for each tracked vehicle	66.39%	Moderate stability; minor variations occurred due to perspective and frame rate differences.
Speed Smoothness	Measures whether speed readings change smoothly across frames	73.77%	Indicates good temporal smoothness; vehicle motion is consistent across frames.
Active Movement	Percentage of vehicles showing continuous motion	38.93%	Some vehicles remained static or moved briefly (e.g., at intersections or lights).
Logical Speed Ranges	Checks that all measured speeds fall within realistic values (0–180 km/h)	99.18%	Almost all detected speeds are within logical physical limits.
Estimated Accuracy	Combined weighted score of all metrics	77.34%	Represents the overall algorithmic performance across detection, tracking, and speed estimation.

Violation Type	Count	Interpretation
Speed Violations	9	Nine vehicles were detected exceeding the defined speed limit (80 km/h).
Lane Violations	1	1 vehicle were identified moving into the opposite lane or crossing defined lane boundaries.

These results confirm that the algorithm correctly identifies and records both types of violations and outputs them in a structured CSV report for further analysis.

6.4 Result Interpretation

The system's **overall estimated accuracy of 77.34%** indicates good performance considering the complexity of outdoor driving environments, camera angles, and varying vehicle sizes.

Key insights include:

- The YOLOv8 model provides **high detection recall**, ensuring nearly all vehicles are identified.
- The **tracking module (ByteTrack)** maintains ID consistency across frames, reducing identity loss.
- Speed estimation accuracy depends on **homography calibration** and **frame rate consistency**.
- Violation detection (speed/lane) performs reliably under clear lighting and moderate traffic density.

Despite environmental noise, the system successfully identifies real-world violations and outputs both visual and statistical evidence.

6.5 Example Output Interpretation

- The “Smart Video Accuracy Report” demonstrates balanced weighting among **stability, smoothness, movement, and logic**—ensuring the final score (77.34%) reflects real-world system reliability.
 - The “Violation Report” validates system usability for **traffic enforcement** applications where automatic identification of overspeeding and lane misuse is required.
-

6.6 Summary

The developed system successfully:

- Tracked **357 vehicles**,
- Produced **244 valid trajectories**,
- Detected **9 speed violations** and **1 lane violation**,
- Achieved an **estimated accuracy of 77.34%**, demonstrating effective real-time vehicle monitoring and rule enforcement.

• 7) Future Plan:

In the future, we plan to improve and expand the Traffic Violation Detection System to make it more accurate and useful in real-life applications.

Some possible extensions include:

- Adding more types of violations as not wearing seat belts.
- Integrating license plate recognition (ANPR) to identify the vehicle and record its number automatically.
- Using real-time video streams from street cameras instead of recorded videos. These improvements will make the system more advanced and help in creating a fully automated smart traffic monitoring solution